

Semantic Duplicate and Near-Plagiarism Detection Engine

Technical Report

Data Mining Project 3

July 5, 2026

1 Overview

This repository implements a command-line plagiarism and near-duplicate detection engine. It follows two algorithmic paths: (1) word shingling, MinHash, and locality sensitive hashing (LSH), and (2) TF-IDF weighted SimHash. The goal is a transparent, reproducible implementation rather than a wrapper around ready-made MinHash, LSH, or SimHash libraries.

2 Preprocessing

Each input document is normalized using Unicode NFKC, lowercasing, removal of unnecessary punctuation, whitespace compaction, English and Persian stopwords filtering, and Persian/Arabic character unification. The normalized text is tokenized and converted to word shingles. The default shingle size is 3. Empty texts, very short texts, and unusual characters are handled without crashing: empty documents produce an empty shingle set, while very short documents become one short shingle.

3 Exact Jaccard Similarity

For two shingle sets A and B , exact similarity is computed as:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}.$$

Computing exact Jaccard for every pair in a collection of n documents requires $n(n-1)/2$ comparisons, which is $O(n^2)$. This is the motivation for MinHash and LSH.

4 MinHash

MinHash signatures are implemented from scratch. Each shingle is mapped to a stable 64-bit hash. A deterministic family of universal hash functions is then applied to compute the minimum value for every signature row. The default signature length is 128. The approximate similarity of two documents is the fraction of equal positions in their signatures.

5 LSH Candidate Generation

The MinHash signature is split into bands. Each band is hashed into a bucket. Two documents become candidates when they share at least one bucket in at least one band. This avoids direct comparison of all pairs. On the included sample corpus, LSH reduces comparisons from 21 all-pairs comparisons to 3 candidate comparisons, a reduction ratio of approximately 0.857.

6 TF-IDF Weighted SimHash

For the second path, each document is represented by token counts. Smoothed IDF is computed over the corpus or pair being evaluated. For every token, the TF-IDF weight is added or subtracted from a 64-dimensional vector according to the 64-bit token hash. The final bit is set to one if the corresponding vector dimension is non-negative. Similar documents should have low Hamming distance and therefore high similarity $1 - d_H/64$.

7 CLI Commands

The project exposes three required commands:

1. **compare**: compare two text files and write JSON scores.
2. **corpus**: search a folder of documents using MinHash+LSH candidates and write candidate pairs.
3. **pairs**: evaluate labelled document pairs and write metrics.

8 Dataset

The repository includes a small reproducible educational corpus under `data/sample_corpus/` and a small labelled pair CSV under `data/raw/train.csv`. The code is also ready for larger datasets such as PAN-PC-11, Stack Exchange duplicate pairs, and Quora Question Pairs. Large raw datasets should remain under `data/raw/`, which is ignored by git.

9 Experimental Results

The following results are generated by the included labelled sample pairs using default parameters: shingle size 3, 128 MinHash permutations, 32 LSH bands, MinHash threshold 0.25, and SimHash threshold 0.75.

Method	Acc.	Prec.	Recall	F1	TP	FP	TN	FN
MinHash_similarity_threshold	0.900	1.000	0.800	0.889	4	0	5	1
TFIDF_weighted_SimHash	0.800	1.000	0.600	0.750	3	0	5	2

10 Error Analysis

The sample data intentionally includes both almost copied texts and paraphrased texts. False negatives mainly occur when a pair is semantically similar but does not preserve enough contiguous word shingles, or when different important terms lead to different SimHash bits. Examples from the current run:

- Pair 3: label=1, MinHash=0.328, SimHash=0.594. The text is semantically related but has limited exact shingle overlap or different key tokens.
- Pair 4: label=1, MinHash=0.078, SimHash=0.719. The text is semantically related but has limited exact shingle overlap or different key tokens.

11 Reproducibility

Install and run:

```
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt -e .
python -m plagiarism_engine.cli compare \
    --file-a data/sample_corpus/doc_01.txt \
    --file-b data/sample_corpus/doc_02.txt \
    --output outputs/two_file_compare.json
python -m plagiarism_engine.cli corpus \
    --data data/sample_corpus \
    --threshold 0.25 \
    --shingle-size 3 \
    --output outputs/candidates.csv
python -m plagiarism_engine.cli pairs \
    --pairs data/raw/train.csv \
    --text-col-a question1 \
    --text-col-b question2 \
    --label-col is_duplicate \
    --limit 5000 \
    --output outputs/metrics.csv
pytest -q
```

12 Conclusion

The implementation satisfies the project requirements with modular source files, a command-line interface, explicit preprocessing, from-scratch MinHash and SimHash, LSH candidate reduction, reproducible outputs, unit tests, and a technical report. The main limitation is that the bundled data is small; for final course submission, it is recommended to run the same commands on PAN-PC-11, Stack Exchange duplicates, or Quora Question Pairs when dataset access is available.